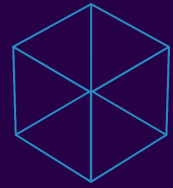




СЕТИ РАДИАЛЬНО- БАЗИСНЫХ ФУНКЦИЙ



Д е р е з а Д а н и л Г е н н а д ь е в и ч



ОБЛАСТИ ПРИМЕНЕНИЯ

АППРОКСИМАЦИЯ И РЕГРЕССИЯ

01

КЛАССИФИКАЦИЯ

02

УПРАВЛЕНИЕ И РОБОТОТЕХНИКА

03

ОБРАБОТКА СИГНАЛОВ ИЗОБРАЖЕНИЙ

04

БИОИНФОРМАТИКА И МЕДИЦИНА

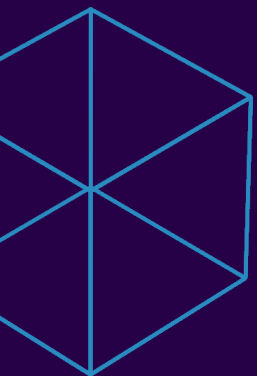
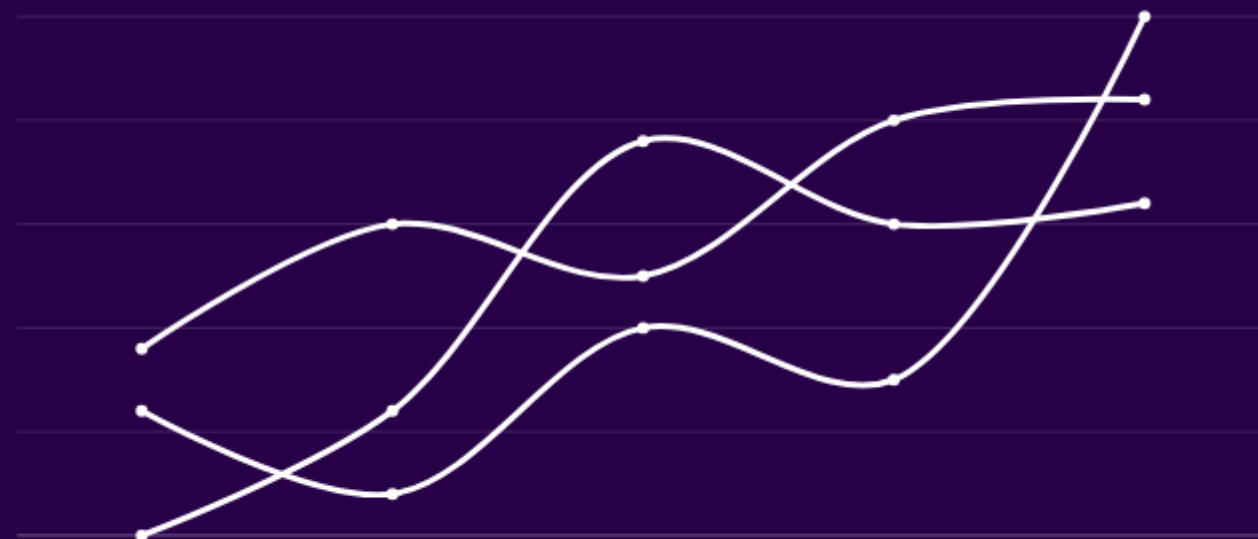
05

ГЕОСТАТИКА И ГЕОФИЗИКА

06

ФИНАНСЫ И ЭКОНОМИКА

07





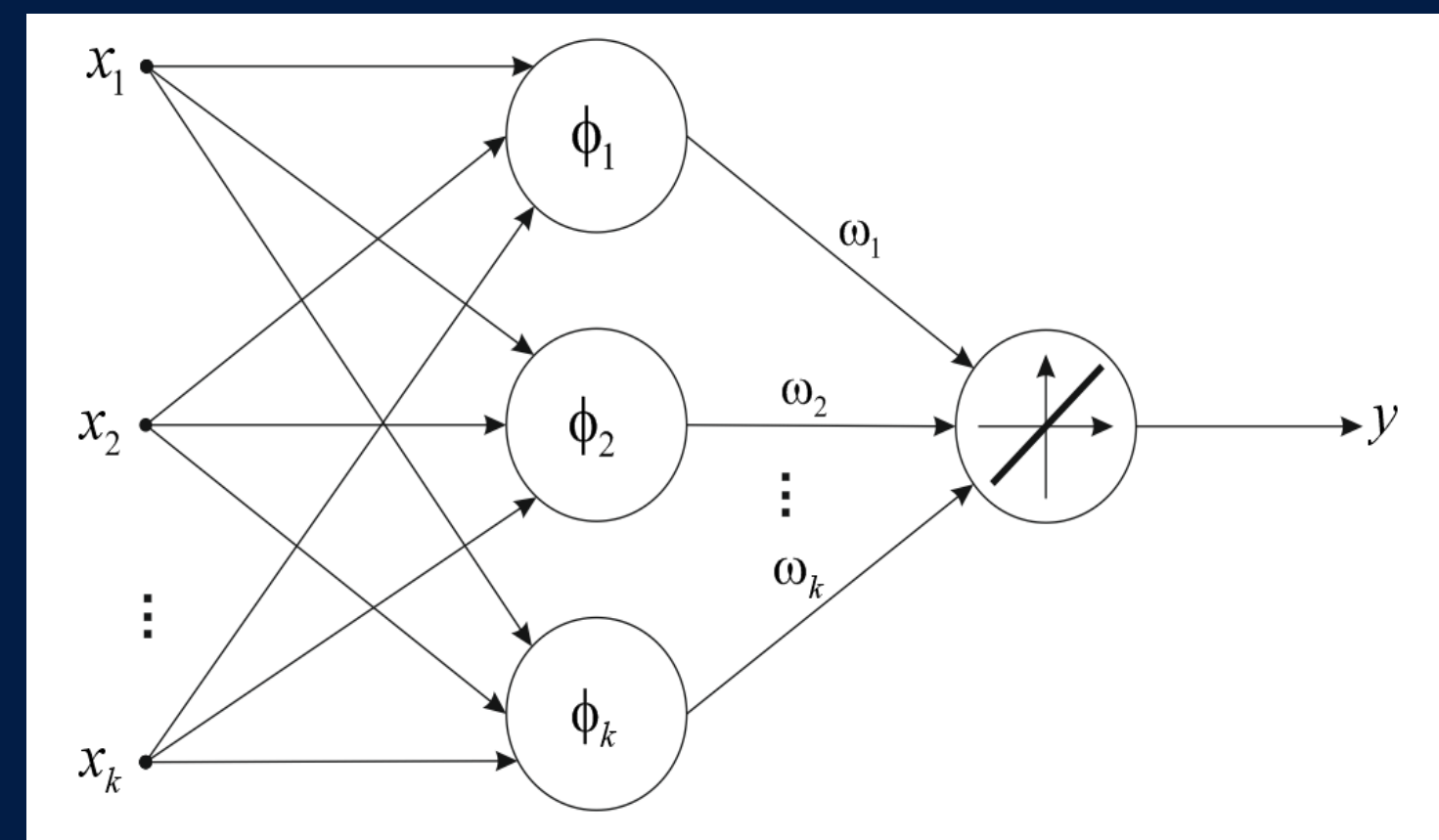
ЧТО ЭТО ТАКОЕ?



02

Сеть радиально-базисных функций – искусственная нейронная сеть, которая использует радиальные базисные функции как функции активации. Выходом сети является линейная комбинация радиальных базисных функций входов и параметров нейрона. Сети радиальных базисных функций имеют множество применений, в том числе функции приближения, прогнозирования временных рядов, классификации и системы управления.

<https://ru.wikipedia.org/...>





ЧТО ЭТО ТАКОЕ?

```

Begin Radial Basis Function (RBF) network input-output demo

Creating a 3-4-2 radial net

Setting centroids (means) to:
-3.00 -3.50 -3.80
-1.00 -1.50 -1.80
 2.00  2.50  2.80
 4.00  4.50  4.80

Loading centroids into radial net

Setting standard deviations (widths) to:
 2.22  3.33  4.44  5.55

Loading standard deviations into radial net

Setting hidden-output weights to:
 5.00 -5.10
-5.20  5.30
-5.40  5.50
 5.60 -5.70

Loading hidden-output weights into radial net

Setting output biases to:
 7.0  7.1

Loading output biases into radial net

Setting x-input to:
 1.0 -2.0  3.0

Computing the output of the radial net

Hidden[0] distance = 8.0306
Hidden[0] output = 0.0014

Hidden[1] distance = 5.2240
Hidden[1] output = 0.2921

Hidden[2] distance = 4.6141
Hidden[2] output = 0.5828

Hidden[3] distance = 7.3817
Hidden[3] output = 0.4129

The output of the RBF network is:
 4.6535  9.4926

End RBF network demo

```

ЦЕНТРОИДЫ

Определяют "зону влияния" каждого скрытого нейрона

01

Чем ближе входной вектор к центроиду, тем сильнее активируется соответствующий RBF-нейрон

02

ШИРИНА (МАСШТАБ)

Определяет, насколько быстро убывает влияние нейрона при удалении от центроида

Малое σ → узкий "колокол" (чувствительность к локальным изменениям)

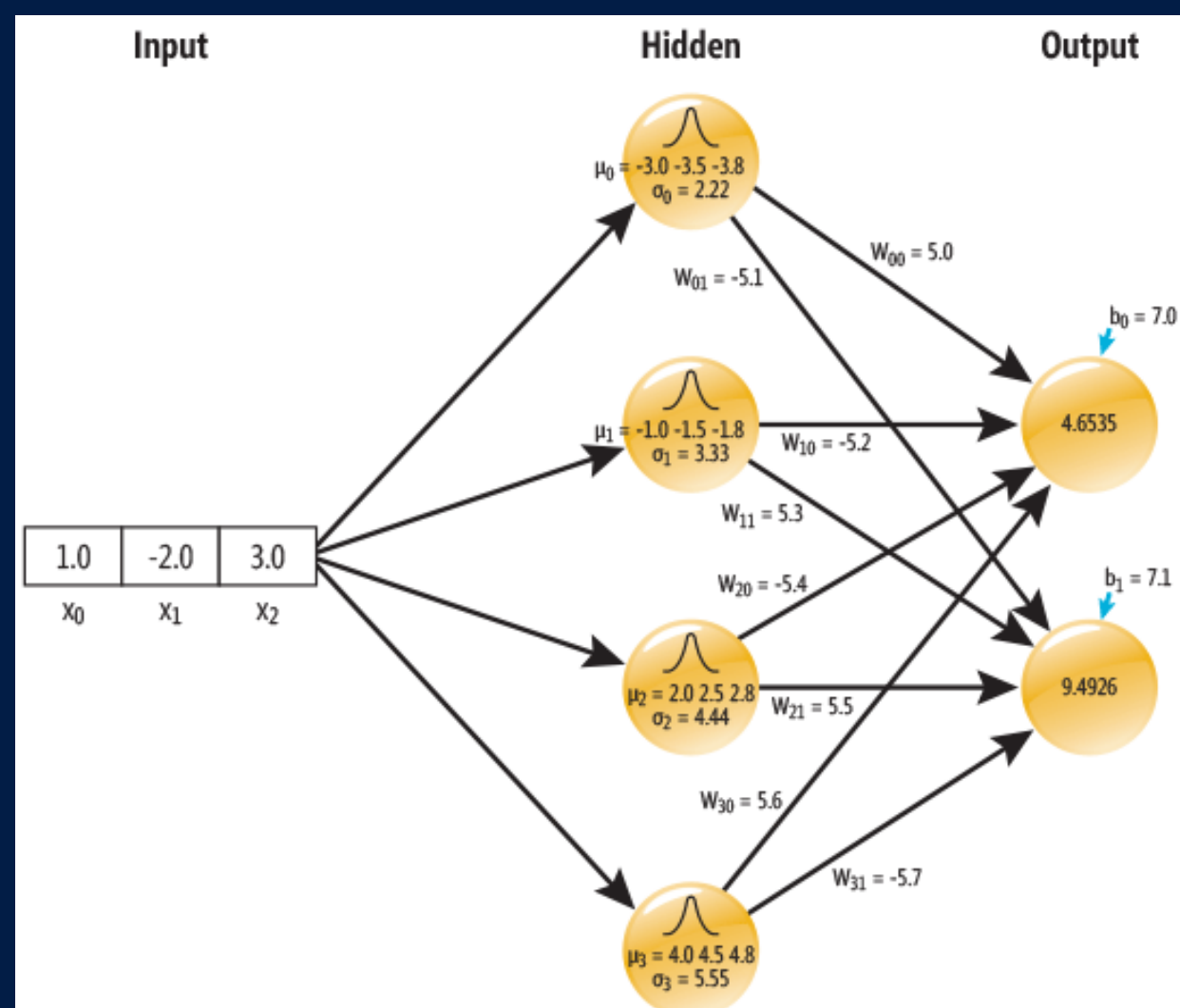
01

Большое σ → широкий "колокол" (сглаживание)

02

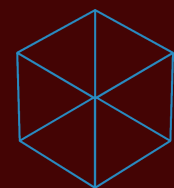
ДЕМОНСТРАЦИЯ

ЭТАПЫ ОБРАБОТКИ:



Входной вектор отправляется в каждый скрытый узел, и каждый скрытый узел независимо вычисляет промежуточное выходное значение

Промежуточные выходные значения скрытых узлов объединяются для получения окончательных выходных значений системы



ОСНОВНЫЕ ВИДЫ RBF

ГАУССОВА
(GAUSSIAN) RBF

$$\varphi(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{c}\|^2}{2\sigma^2}\right), \sigma > 0$$

МУЛЬТИКВАДРАТИЧНАЯ
(MULTIQUADRATIC) RBF

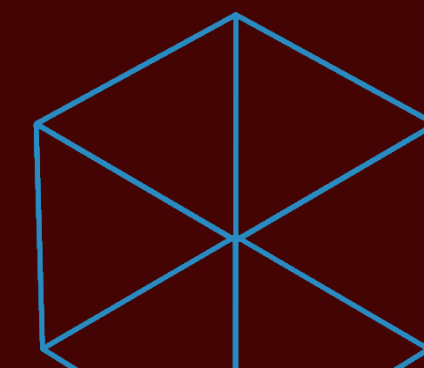
$$\varphi(\mathbf{x}) = \sqrt{\|\mathbf{x}-\mathbf{c}\|^2 + c^2}, c > 0$$

ОБРАТНАЯ
МУЛЬТИКВАДРАТИЧНАЯ
(INVERSE MULTIQUADRATIC) RBF

$$\varphi(\mathbf{x}) = \frac{1}{\sqrt{\|\mathbf{x}-\mathbf{c}\|^2 + c^2}}, c > 0$$

МОДИФИЦИРОВАННАЯ
ТОНКОПЛАСТИНЧАТАЯ
СПЛАЙНОВАЯ (MODIFIED
THIN-PLATE SPLINE) RBF

$$\varphi(\mathbf{x}) = \left(\frac{\|\mathbf{x}-\mathbf{c}\|}{\sigma}\right)^2 \ln\left(\frac{\|\mathbf{x}-\mathbf{c}\|}{\sigma}\right), \sigma > 0$$





ЭТАПЫ ОБУЧЕНИЯ

08

НЕУПРАВЛЯЕМОЕ ОБУЧЕНИЕ

Цель: Найти оптимальные параметры скрытого слоя (c и σ)

ОПРЕДЕЛЕНИЕ ЦЕНТРОВ

01

Основной метод: K-means

ОПРЕДЕЛЕНИЕ ШИРИНЫ

02

Основные методы:

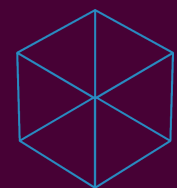
Правило ближайшего соседа

Глобальная ширина (одинаковая для всех центров)

$$\sigma_i = \frac{1}{M} \sum_{j=1}^M ||c_i - c_j||$$

$$\sigma = \frac{d_{max}}{\sqrt{2K}}$$





ПРИМЕРЫ



НЕУПРАВЛЯЕМОЕ ОБУЧЕНИЕ

ОПРЕДЕЛЕНИЕ ЦЕНТРОВ

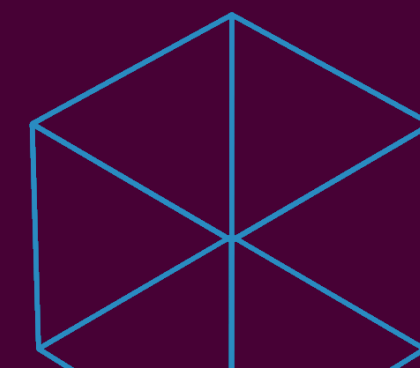
01

```
def compute_centers(X, n_centers=10, random_state=42):  
    if isinstance(X, torch.Tensor):  
        X_np = X.numpy()  
    else:  
        X_np = X  
  
    kmeans = KMeans(n_clusters=n_centers, random_state=random_state)  
    kmeans.fit(X_np)  
  
    centers = torch.tensor(kmeans.cluster_centers_, dtype=torch.float32)  
    return centers
```

ОПРЕДЕЛЕНИЕ ШИРИНЫ

02

```
def compute_sigmas(centers, m=2):  
    dists = torch.cdist(centers, centers)  
    dists.fill_diagonal_(float('inf'))  
    sigmas = torch.mean(torch.topk(dists, m, largest=False).values, dim=1)  
    return sigmas
```





ЭТАПЫ ОБУЧЕНИЯ

УПРАВЛЯЕМОЕ ОБУЧЕНИЕ

Цель: Найти веса выходного слоя, минимизирующие ошибку.

Основные методы:

Линейная регрессия (псевдообратная матрица)

01

Градиентный спуск (если требуется нелинейная оптимизация)

02





ЭТАПЫ ОБУЧЕНИЯ

ЛИНЕЙНАЯ РЕГРЕССИЯ

Выход сети: $y = W \cdot \Phi$, где
 Φ – выходы RBF-функций,
 y – предикты

Решение в матричной форме: $W = Y \cdot \Phi^+$, где

Y – таргетные значения из обучающих данных,
 Φ^+ – псевдообратная матрица Мура–Пензоуза для Φ

$\Phi = U \cdot S \cdot V^T \Rightarrow \Phi^+ = V \cdot S^+ \cdot U^T$, где

S^+ – диагональная матрица с обратными значениями ненулевых
сингулярных чисел $\sigma_i = \sqrt{\lambda_i}$, где

λ_i – собственное значение матрицы $\Phi^T \Phi$





ЭТАПЫ ОБУЧЕНИЯ

ГРАДИЕНТНЫЙ СПУСК

Среднеквадратичная ошибка:

$$E = \frac{1}{2} \sum_{i=1}^N \left(y_{target}^{(i)} - y_{pred}^{(i)} \right)^2$$

01

Корректировка весов:

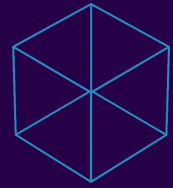
$$w_j^{new} = w_j^{old} - \eta \cdot \frac{\partial E}{\partial w_j}, \text{ где}$$

02

η – learning rate

$\frac{\partial E}{\partial w_j}$ – градиент по весу w_j





ПРИМЕРЫ

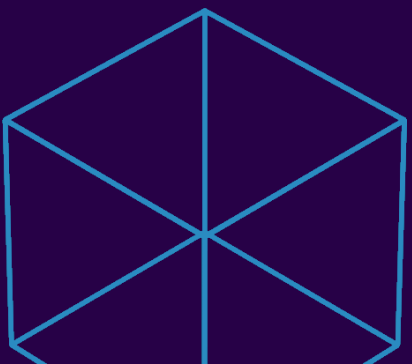


ЛИНЕЙНАЯ РЕГРЕССИЯ

```
def linear_regression_fit(X, y, centers, sigmas):  
    rbf_layer = RBFLayer(centers, sigmas)  
    phi = rbf_layer(X)  
    W = torch.linalg.pinv(phi) @ y.unsqueeze(1)  
    y_pred = phi @ W  
    return W, y_pred
```

ГРАДИЕНТНЫЙ СПУСК

```
def gradient_descent_fit(X, y, centers, sigmas, n_epochs=100, lr=0.1, print_every=10):  
    model = RBFNet(centers, sigmas)  
    criterion = torch.nn.MSELoss()  
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)  
  
    losses = []  
    for epoch in range(n_epochs):  
        optimizer.zero_grad()  
        outputs = model(X)  
        loss = criterion(outputs, y.unsqueeze(1))  
        loss.backward()  
        optimizer.step()  
        losses.append(loss.item())  
  
        if print_every and epoch % print_every == 0:  
            print(f'Epoch {epoch:3d}/{n_epochs}, Loss: {loss.item():.6f}')  
  
    with torch.no_grad():  
        W = model.linear.weight.detach().t() # (n_centers, 1)  
        y_pred = model(X)  
  
    return W, y_pred, losses
```





ДОСТОИНСТВА RBF-СЕТЕЙ

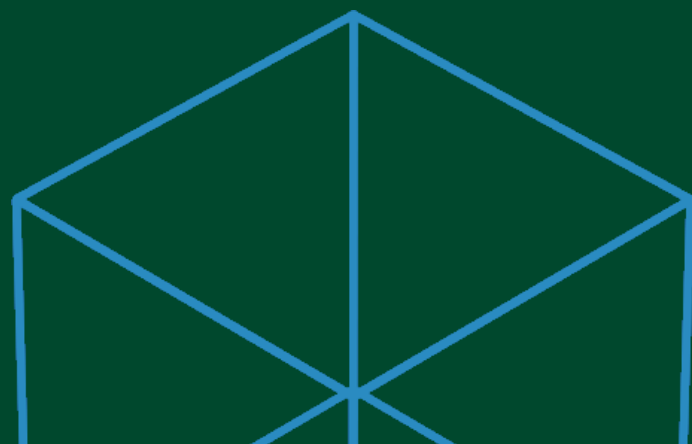
ОТНОСИТЕЛЬНО БЫСТРОЕ ОБУЧЕНИЕ	01
УНИВЕРСАЛЬНАЯ АППРОКСИМАЦИЯ	02
ИНТЕРПРЕТИРУЕМОСТЬ	03
УСТОЙЧИВОСТЬ К ШУМУ	04
ХОРОШИЕ РЕЗУЛЬТАТЫ НА МАЛЫХ ДАННЫХ	05





НЕДОСТАТКИ RBF-СЕТЕЙ

ПРОБЛЕМА "ПРОКЛЯТИЯ РАЗМЕРНОСТИ"	01
ЗАВИСИМОСТЬ ОТ ВЫБОРА ЦЕНТРОВ И ШИРИНЫ	02
ПЛОХАЯ МАСШТАБИРУЕМОСТЬ	03
СЛОЖНОСТЬ В НАСТРОЙКЕ ГИПЕРПАРАМЕТРОВ	04
ОГРАНИЧЕННАЯ СПОСОБНОСТЬ К ОБОБЩЕНИЮ	05





СПОСОБЫ УЛУЧШЕНИЯ RBF-СЕТЕЙ

УСКОРЕНИЕ ОБУЧЕНИЯ

ОПТИМИЗАЦИЯ КЛАСТЕРИЗАЦИИ

01

Mini-batch k-means – обработка данных частями ($O(B \cdot k \cdot d)$ вместо $O(n \cdot k \cdot d)$)

KD-деревья – ускорение поиска ближайших центров

АППРОКСИМАЦИЯ МАТРИЦ

02

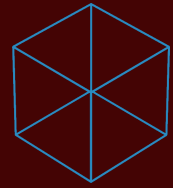
Рандомизированный SVD – ускорение псевдообращения для больших k

СТОХАСТИЧЕСКАЯ ОПТИМИЗАЦИЯ ВЕСОВ

03

Градиентный спуск с mini-batch – обновление весов на лету без пересчёта центров





СПОСОБЫ УЛУЧШЕНИЯ RBF-СЕТЕЙ

ПОВЫШЕНИЕ ТОЧНОСТИ

ГИБРИДНЫЕ МОДЕЛИ

RBF + SVM

RBF как слой в DNN

01

ОПТИМИЗАЦИЯ ГИПЕРПАРАМЕТРОВ

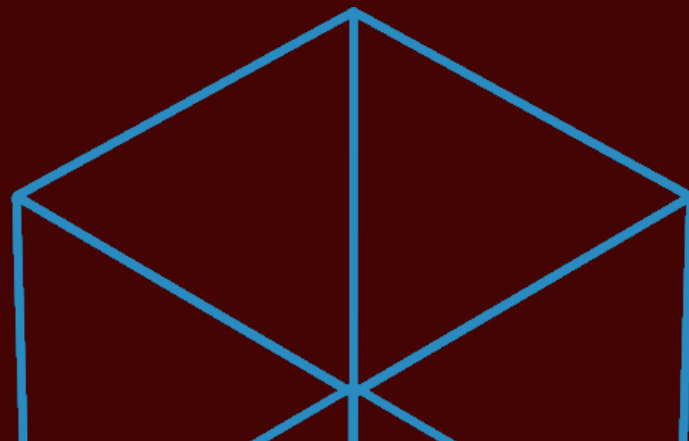
GridSearch/RandomSearch

02

АНСАМБЛИ

Bagging RBF-сетей – усреднение предсказаний нескольких моделей с разными центрами

03





СПАСИБО ЗА ВНИМАНИЕ

